

Quando usar?

- Quando há múltiplos objetos que precisam se comunicar entre si.
- Quando deseja reduzir dependências diretas entre classes.
- Quando a lógica de comunicação entre objetos se torna complexa.

Benefícios

1. **Desacoplamento:** Reduz dependências diretas entre objetos.
2. **Centralização:** Toda a lógica de comunicação é gerenciada pelo mediador.
3. **Facilidade de Manutenção:** Alterar a comunicação entre objetos impacta apenas o mediador.

Exemplo Prático

Cenário

Em um sistema de **chat**, vários usuários podem enviar e receber mensagens. Em vez de os usuários se comunicarem diretamente, o **mediador** (um "servidor de chat") gerencia todas as mensagens e distribui para os destinatários corretos.

Implementação

1. Interface do Mediator

Define o contrato para comunicação.

```
public interface IChatMediator
{
    void RegisterUser(ChatUser user);
    void SendMessage(string message, ChatUser sender);
}
```

2. Implementação do Mediator

Gerencia os usuários e distribui mensagens.

```
public class ChatMediator : IChatMediator
{
    private readonly List<ChatUser> _users = new List<ChatUser>();

    public void RegisterUser(ChatUser user)
    {
        _users.Add(user);
    }

    public void SendMessage(string message, ChatUser sender)
    {
        foreach (var user in _users)
        {
            if (user != sender)
            {
                user.ReceiveMessage(message, sender);
            }
        }
    }
}
```

3. Classe Base para os Usuários

Define o comportamento comum entre os usuários.

```
public abstract class ChatUser
{
    protected IChatMediator Mediator;
    public string Name { get; }

    protected ChatUser(IChatMediator mediator, string name)
    {
        Mediator = mediator;
        Name = name;
    }

    public abstract void SendMessage(string message);
    public abstract void ReceiveMessage(string message, ChatUser sender);
}
```

4. Implementação dos Usuários

Cada usuário usa o mediador para enviar mensagens.

```
public class User : ChatUser
{
    public User(IChatMediator mediator, string name) : base(mediator,
name) { }

    public override void SendMessage(string message)
    {
        Console.WriteLine($"{Name} sends: {message}");
        Mediator.SendMessage(message, this);
    }

    public override void ReceiveMessage(string message, ChatUser sender)
    {
        Console.WriteLine($"{Name} received from {sender.Name}:
{message}");
    }
}
```

5. Uso do Mediator

```
class Program
{
    static void Main(string[] args)
    {
        // Criar o mediador
        IChatMediator chatMediator = new ChatMediator();

        // Criar usuários e registrá-los no mediador
        var user1 = new User(chatMediator, "Alice");
        var user2 = new User(chatMediator, "Bob");
        var user3 = new User(chatMediator, "Charlie");

        chatMediator.RegisterUser(user1);
        chatMediator.RegisterUser(user2);
        chatMediator.RegisterUser(user3);

        // Enviar mensagens
        user1.SendMessage("Hello, everyone!");
        user2.SendMessage("Hi, Alice!");
        user3.SendMessage("Good morning!");
    }
}
```

Saída Esperada

```
Alice sends: Hello, everyone!
Bob received from Alice: Hello, everyone!
Charlie received from Alice: Hello, everyone!

Bob sends: Hi, Alice!
Alice received from Bob: Hi, Alice!
Charlie received from Bob: Hi, Alice!

Charlie sends: Good morning!
Alice received from Charlie: Good morning!
Bob received from Charlie: Good morning!
```

Vantagens do Mediator Pattern

1. **Redução de Dependências:**
 - Os objetos se comunicam através do mediador, e não diretamente entre si.
2. **Centralização da Lógica:**
 - Facilita o controle e a modificação das interações.
3. **Flexibilidade:**
 - Novos objetos podem ser adicionados sem alterar os já existentes.

Quando evitar?

- Se o mediador crescer muito em complexidade, ele pode se tornar um "ponto único de falha".
- Se o número de interações for simples, o padrão pode ser um exagero.

Resumo

O **Mediator Pattern** é ideal para sistemas onde muitos objetos precisam interagir entre si, mas não devem conhecer uns aos outros diretamente. Ele centraliza a comunicação, melhorando a modularidade e a escalabilidade do sistema. □